Experimental Methods in Computer Science

Experiment Design

André Miguel Correia Cardoso 2022222265

Luis Miguel Ferreira Vaz 2022214707

Pedro Costa 2022220304

University of Coimbra, Mestrado em Engenharia Informática 2025/2026

Contents

1 Introduction **4**

2 Bug Finding **4**

2.1 Experimental Design 4

2.1.1 Research Question 4

2.1.2 Independent Variables 5

2.1.3 Dependent Variables 5

2.1.4 Controlled Variables 5

2.2 Experimental Procedure 6

2.2.1 Dataset & Data Collection 6

2.2.2 Prompt Building 6

2.2.3 Submission Mode 6

2.2.4 Response Classification & Metrics Classification 7

2.3 Results 7

2.4 Statistical Analysis 7

3 Code Generation **8**

3.1 Experimental Design 8

3.1.1 Research Questions 8

3.1.2 Independent Variables 9

3.1.3 Dependent Variables 9

3.1.4 Controlled Variables 10

3.2 Experimental Procedure 10

3.2.1 Dataset & Data Collection 10

3.2.2 Query Generation Procedure 11

3.2.3 Evaluation Metrics 11

3.3 Results 11

3.3.1 Descriptive Statistics 12

3.3.2 Distribution by Difficulty 12

3.3.3 Per-Problem Comparison 14

3.4 Statistical Analysis 15
3.4.1 Model's Correctness Comparison 15
3.4.2 Model's Execution Time Comparison 16
3.4.3 Impact of Problem Difficulty on Correctness 19

1 Introduction

Large Language Models (LLMs) are increasingly being integrated into software development workflow. As their use becomes more widespread, it is important to evaluate the extent to which developers can rely on these tools in practical software engineering contexts.

This report was developed as part of the Experimental Methods in Computer Science curricular unit and addresses the following central problem statement:

Can developers rely on LLM-based tools to support software development activities?

Given the wide scope of this question, the experiment focuses on two specific areas of software development: Code Generation and Code Review (more specifically Bug Finding). These two areas were studied separately, using distinct datasets and statistical analysis methods suited to each task.

Despite these differences, both parts of the experiment share a common setting: they are based on the Python programming language and evaluate the same LLM models. This allows for a structured comparison of model performance across two relevant software development activities, contributing to a broader understanding of the capabilities and limitations of LLM-based tools.

2 Bug Finding

Code Review, and more specifically Bug Finding, refers to the practice of providing an LLM with source code and asking it to inspect the code for potential defects. Depending on the experimental setup, this process may involve supplying only the relevant code files, or it may include additional contextual information such as failing tests, execution outputs, error messages, or brief descriptions of the expected behaviour.

This experiment focuses on the bug detection capabilities of the selected LLMs. In particular, the models are tasked with analysing Python source code and identifying the locations of real bugs. The objective is to evaluate how effectively each model can support developers in detecting defects during code review activities.

2.1 Experimental Design

2.1.1 Research Question

The research question for this experiment is:
How does the choice of Large Language Model affect bug-finding performance in real-world Python bugs?

2.1.2 Independent Variables

The independent variables are:

- **LLM Model (2 levels)**:
  - ChatGPT Mini
  - Gemini Flash
- **LOC (3 levels)**: LOC (*Lines of Code*) of the source files.
  - Low LOC level — ≤ 330 LOC
  - Medium LOC level — 331–980 LOC
  - High LOC level — >980 LOC

2.1.3 Dependent Variables

The dependent variables used in the statistical comparison are:

- **Recall**: Fraction of the real bugs identified.

$$Recall = \frac{TP}{TP + FN}$$

- **F1-Score**: F1-score combines recall and precision into a single metric that penalises both missed bugs and incorrectly reported bugs:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

Precision was excluded, since, due to its formula, when an LLM reports zero bugs, it will generate a division by 0. Since F1-score is the harmonic mean between Recall and Precision we decided to opt for that dependent variable.

2.1.4 Controlled Variables

In order to reduce unnecessary variables and isolate each LLM's performance, we kept the following variable constant:

- **Scenarios**: Both LLMs are fed the same bug scenarios
- **Prompt**: All prompts follow the same format. They just vary from scenario to scenario in the Source Files section, since they include that specific scenario's buggy code.
- **New chat for each submission**: In order to avoid outputs based on previous context, each submission is made in a new chat.
- **Language (1 level)** :

– Python

Even though there is only one language bing used in the bug, it is still considered an independent variable.

2.2 Experimental Procedure

2.2.1 Dataset & Data Collection

To obtain a realistic dataset of Python bugs, the BugsInPy repository was used. A total of 50 bug scenarios were collected, each corresponding to a real defect extracted from an open-source Python project.
For each scenario, the dataset includes the buggy source code, the associated Git diff identifying the modified buggy chunk or chunks, the failing test file, and the failing test output. This information provides both the defective implementation and the contextual evidence needed to analyse the bug, while preserving the realism of real-world software. defects

2.2.2 Prompt Building

To ensure consistency across all bug scenarios, a standard prompt template was created and stored in(`/BugFinding/LLM_prompt.txt`). This template was used as the basis for each scenario-specific prompt, ensuring that both LLMs received instructions in the same format and under comparable conditions.

Each prompt begins by defining the task as bug localization only, explicitly instructing the model to identify the source-code location or locations most likely responsible for the failing test. The prompt also includes rules designed to constrain the model's behaviour: the model is instructed not to propose fixes, not to rewrite code, and not to provide explanations outside the required output format.

The expected response format is specified as valid JSON, containing a list of suspected bug locations.

After the general instructions, each prompt includes the buggy source file or files, annotated with line numbers so that the model can refer to precise locations in its response. The failing test file is also provided, together with the corresponding test execution output. All of this information is combined into a single `.txt` file for each bug scenario, allowing the model to analyse the source code, the test logic, and the observed failure within the same prompt.

2.2.3 Submission Mode

Each bug scenario was submitted in a separate conversation to minimise the influence of previous interactions on the model's responses. This was done to avoid hysteresis, where earlier prompts or outputs may influence the model's behaviour in later scenarios.

Each scenario-specific prompt was submitted using the file attachment functionality rather than being manually pasted into the conversation. This reduced the risk of formatting changes, missing content, or accidental text deformation during input, helping to preserve the structure of the prompt exactly as intended.

2.2.4 Response Classification & Metrics Classification

For both LLM models, the outputs were stored under JSON format in the `BugFinding/ChatGPT_outputs` and `BugFinding/Gemini_outputs` directories. The benchmarks for the bug locations were also stored under JSON format in the `BugFinding/benchmarks directory`.

The script `BugFinding/compare_bug_locations_folder.py` is used to calculate the metrics for each output. The script compares the reported bug locations with the benchmark locations. Then the script calculates the number of true positives, false positives and false negatives for each case. Additionally, the precision, recall and F1-score are calculated as well.

2.3 Results

In the table below, the total overall results can be viewed for both LLM models.

Model	**Num of Cases**	**Num of real bugs**	**Num reported bugs**	**TP**	**FP**	**FN**
ChatGPT	106	85	94	57/60		
Gemini	103	83	79	56/60		

Table 1: Descriptive statistics of execution time for accepted submissions.

2.4 Statistical Analysis

To assess the normality of the F1-Score distribution for each model, Q-Q plots were generated and Shapiro-Wilk tests were conducted.

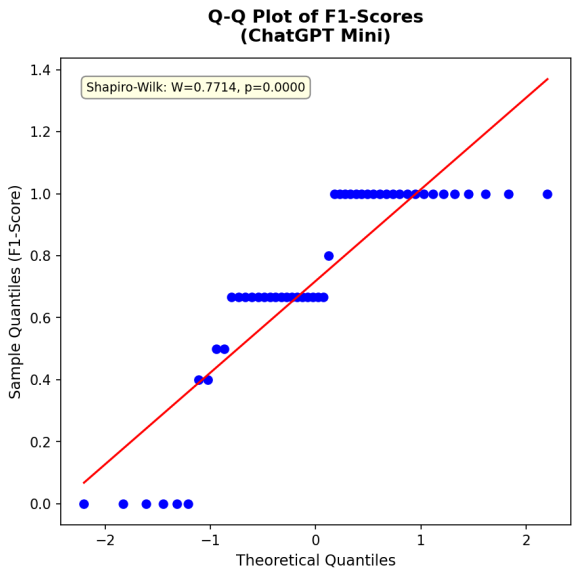

Figure 1: Q-Q plot of F1-Scores for ChatGPT Mini.

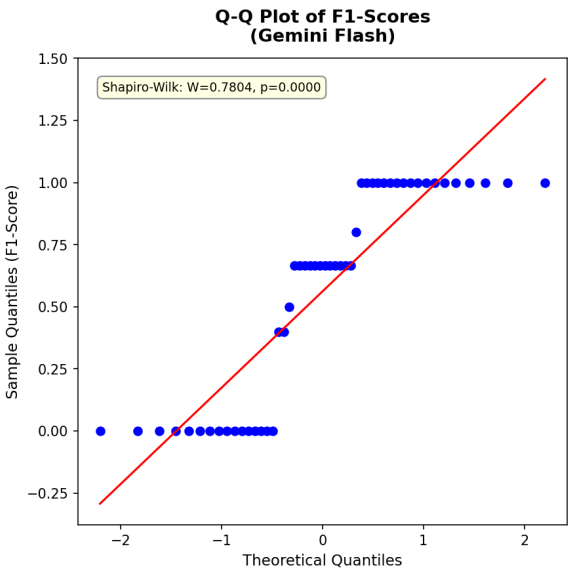

Figure 2: Q-Q plot of F1-Scores for Gemini Flash.

3 Code Generation

Code Generation refers to the task of generating source code from a natural language specification. In the context of this experiment, the selected Large Language Models (LLMs) were required to generate SQL queries that solve programming problems obtained from the LeetCode platform.

SQL query generation was selected because it provides a standardized and objectively verifiable setting in which generated solutions can be automatically executed and validated against predefined requirements. This allows the reliability of the generated code to be assessed without the need for subjective evaluation.

The objective of this experiment is to evaluate the reliability of the selected LLMs in generating correct SQL solutions and to determine whether problem difficulty influences their performance.

3.1 Experimental Design

To ensure experimental consistency, all problems were presented using the same prompt structure. Only the problem statement and input/output specifications varied between tasks.

Furthermore, each problem was submitted in an independent conversation context, preventing information from previous interactions from influencing subsequent responses. As a result, every generated solution constitutes an independent observation.

3.1.1 Research Questions

The experiment was designed to investigate the reliability of Large Language Models (LLMs) in SQL code generation. Specifically, the study aims to address the following research questions:

- **RQ1**: Is there a statistically significant difference in the correctness of SQL queries generated by ChatGPT Mini and Gemini Flash?
- **RQ2**: Does problem difficulty significantly influence the correctness of SQL queries generated by Large Language Models?

3.1.2 Independent Variables

IV1: LLM Model

This variable represents the Large Language Model used to generate SQL queries. It has two levels:

- ChatGPT Mini
- Gemini Flash

IV2: Problem Difficulty

This variable represents the difficulty level of the SQL programming problems selected from the dataset. It has three levels:

- Easy
- Medium
- Hard

3.1.3 Dependent Variables

DV1: Query Outcome Classification (Primary Outcome)

This variable classifies the outcome of each generated SQL query based on its execution and correctness. It is defined as a categorical variable with the following levels:

- **Correct**: the query executes successfully and passes all hidden test cases
- **Incorrect Logic**: the query executes successfully but fails one or more hidden test cases due to logical or constraint violations
- **Syntax or Execution Error**: the query fails to execute due to syntax errors or runtime issues

This single dependent variable captures both execution success and semantic correctness, allowing a clear and non-overlapping classification of model output quality.

DV2: Execution Time (Secondary Performance Metric)

This variable records the execution time reported by the evaluation platform for each query. It is treated as a continuous variable and used as a proxy for query efficiency. It is not used as a measure of correctness, but instead as a secondary performance indicator to compare computational efficiency across models.

3.1.4 Controlled Variables

Several factors that could influence the quality of the generated solutions were kept constant throughout the experiment to ensure that observed differences could be attributed to the independent variables under investigation.

CV1: Prompt Template

All problems were presented using the same prompt template (`/CodeGeneration/CodeGen_prompt.txt`). Only the problem statement and the associated input/output specifications varied between tasks.

CV2: Number of Generation Attempts

Each model was allowed a single attempt per problem. No retries, prompt refinements, or iterative interactions were permitted.

CV3: Evaluation Platform

All generated solutions were evaluated using the LeetCode platform under the same evaluation procedure and hidden test cases.

CV4: Conversation Context

Each problem was submitted in an independent conversation context. This ensured that information from previous interactions could not influence subsequent responses.

3.2 Experimental Procedure

3.2.1 Dataset & Data Collection

The dataset consisted of 60 SQL programming problems collected from the LeetCode platform. To ensure a balanced evaluation across different complexity levels, the selected problems were evenly distributed among the Easy, Medium, and Hard difficulty categories defined by the platform.

Each problem was independently submitted to both LLMs, resulting in a total of 120 generated SQL solutions. The problem statement and database schema provided by LeetCode were used as the input context for the models.

3.2.2 Query Generation Procedure

Both models received the same prompt template and were instructed to generate a single SQL solution for each problem. To evaluate first-response reliability, only one generation attempt was allowed per problem.

Each submission was performed in a new conversation instance to eliminate any influence from previous interactions and ensure consistent experimental conditions across all evaluations.

3.2.3 Evaluation Metrics

The generated SQL queries were evaluated using the LeetCode judging system, which executes each solution against a set of hidden test cases and verifies whether the produced output matches the expected results.

Two metrics were collected during the evaluation process:

- **Correctness**: Binary outcome indicating whether the solution was accepted or rejected by the platform.
- **Execution Time**: Runtime reported by LeetCode for accepted solutions, used as an indicator of query efficiency.

Correctness was considered the primary metric, as it directly reflects the reliability of the generated solutions.

3.3 Results

The generated SQL solutions were evaluated on 60 problems per model. Because each problem was submitted once and the platform returned a binary outcome (Accepted or Not Accepted), correctness and accuracy are equivalent in this setting and are computed as:

$$\text{Accuracy} = \frac{\text{Accepted Solutions}}{\text{Total Problems}}$$

Table 2 summarises the overall correctness of both models.

Model	**Accepted**	**Total**	**Accuracy**
ChatGPT	57	60	95.00%
Gemini	56	60	93.33%

Table 2: Overall correctness of the SQL code generation models.

The two models performed very similarly overall, with ChatGPT obtaining a slightly higher acceptance rate. The difference is driven by one additional rejected solution in Gemini's output.

In both cases, the models achieved perfect performance on Easy problems and near-perfect performance on Medium problems.

Table 3 breaks down the results by problem difficulty.

Difficulty	ChatGPT	Gemini	Total Problems
Easy	20/20 (100.00%)	20/20 (100.00%)	20
Medium	20/20 (100.00%)	19/20 (95.00%)	20
Hard	17/20 (85.00%)	17/20 (85.00%)	20

Table 3: Correctness by difficulty level.

The hardest problems produced the lowest acceptance rate for both models, indicating that problem complexity had a clear effect on performance. The Hard category accounts for all three ChatGPT failures and for three of the four Gemini failures. One additional Gemini failure occurred in the Medium category.

Beyond correctness, the execution time reported by the LeetCode platform was recorded for each accepted submission and used as a secondary performance indicator. The following analysis is based exclusively on accepted solutions, as rejected queries do not produce meaningful runtime measurements.

3.3.1 Descriptive Statistics

Table 4 summarises the central tendency and dispersion of execution times across all accepted submissions.

Model	Mean (ms)	Median (ms)	Std. Dev. (ms)	Accepted
ChatGPT	109	82	122	57/60
Gemini	107	82	95	56/60

Table 4: Descriptive statistics of execution time for accepted submissions.

The two models present identical median runtimes (82 ms for both ChatGPT and Gemini), and their means differ by only 2 ms. However, both distributions exhibit considerable right skew, driven by a small number of outliers. Notably, ChatGPT produced a 764 ms result on Problem 1158 (Medium) and Gemini produced results of 644 ms and 445 ms on Problems 577 and 1084 (Easy), respectively. These outliers inflate the mean substantially relative to the median, which is why the median is a more representative measure of central tendency for this dataset.

3.3.2 Distribution by Difficulty

Figure 3 shows the mean execution time grouped by difficulty level and model. ChatGPT presents a higher average on Hard problems, largely due to the 229 ms result on Problem 1890, while

Gemini achieves a lower average on that category. Conversely, Gemini's Easy average is inflated by the two outliers mentioned above.

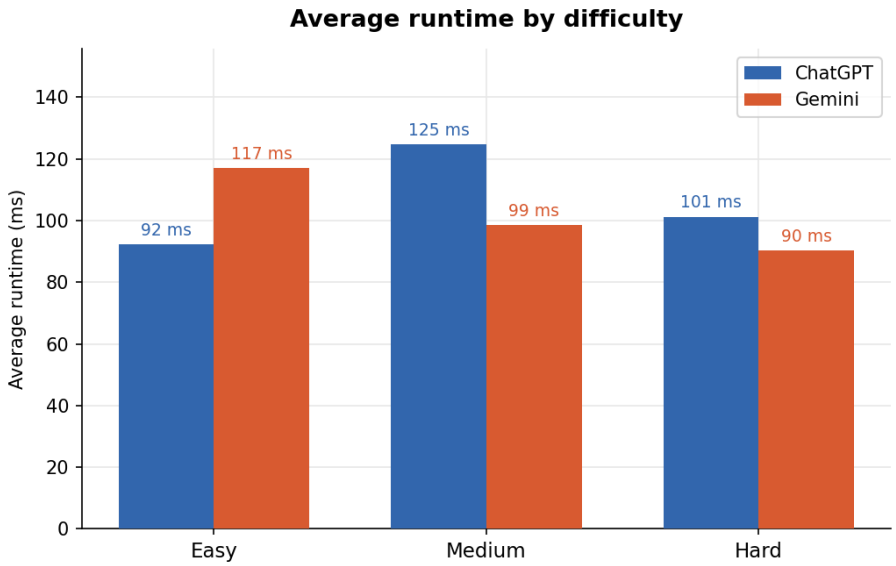

Figure 3: Mean execution time by difficulty level.

The box plots in Figure 4 corroborate this observation. Both models display similar interquartile ranges across difficulty levels, indicating that the majority of queries execute within a comparable and consistent time range. The outliers visible in the Easy category for Gemini are the primary source of distributional asymmetry for that model.

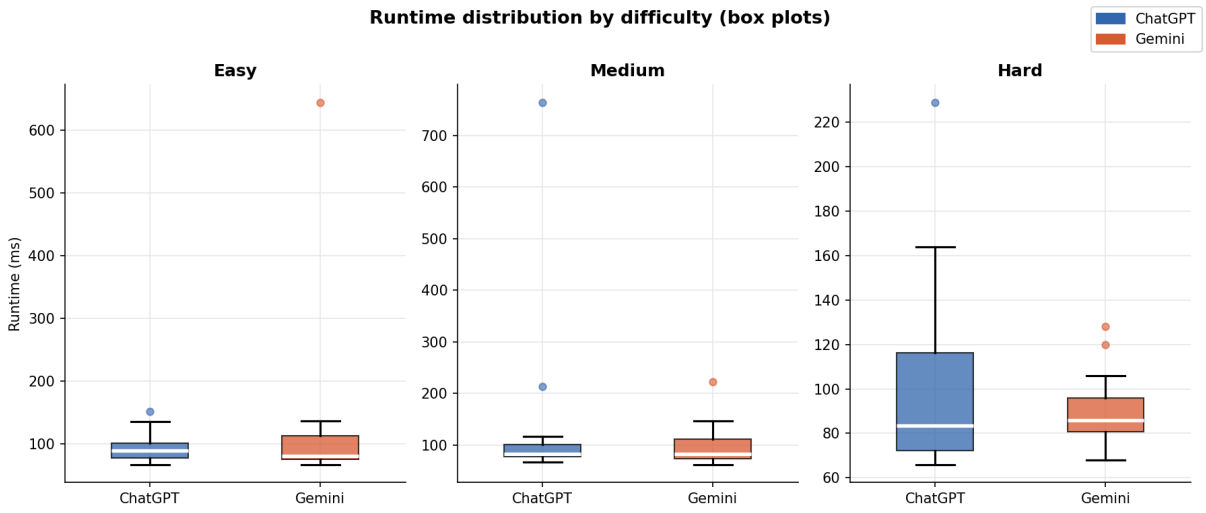

Figure 4: Runtime distribution by difficulty level (box plots). Boxes show the interquartile range; horizontal lines indicate the median; whiskers extend to the minimum and maximum non-outlier values.

3.3.3 Per-Problem Comparison

Figure 5 provides a direct head-to-head comparison of execution times for each problem accepted by both models. No consistent dominance is observed for either model: each outperforms the other on roughly half the shared problems. The largest discrepancies arise from the individual outliers previously identified and are not representative of a systematic performance gap.

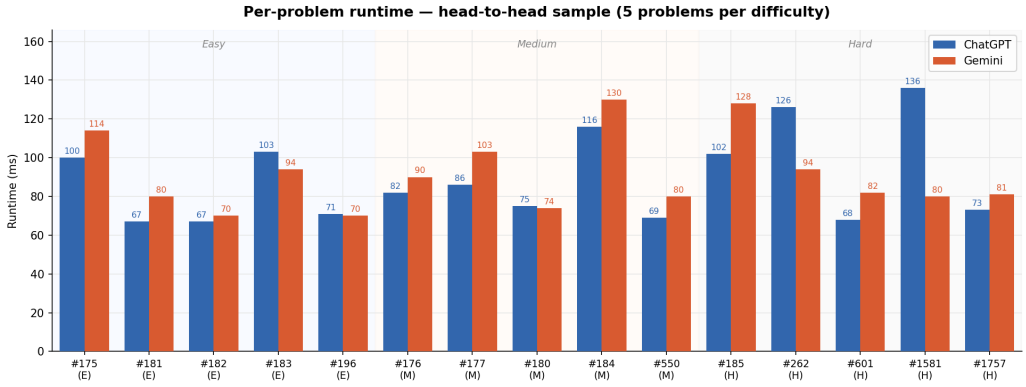

Figure 5: Per-problem execution time comparison for all problems accepted by both models.

3.4 Statistical Analysis

3.4.1 Model's Correctness Comparison

(i) Hypothesis **Null Hypothesis (H_0):** *There is no statistically significant difference in the correctness of SQL queries generated by ChatGPT Mini and Gemini Flash.*

Alternative Hypothesis (H_1):
There is a statistically significant difference in the correctness of SQL queries generated by ChatGPT Mini and Gemini Flash.

(ii) Statistical Test Selection

Since both models were evaluated on the same set of problems and the outcome is binary (Accepted or Not Accepted), the paired comparison is appropriately evaluated using McNemar's test.

Data Assumption Checks:

- **Paired nominal data**: Met. Each problem acts as its own control, producing a binary outcome for both ChatGPT and Gemini.
- **Discordant pairs**: In a paired test, concordant pairs (where both models agree) do not help differentiate performance. McNemar's test exclusively analyzes discordant pairs (instances where the models disagree, i.e., one is correct and the other incorrect) to determine if one model consistently wins these "tie-breakers". Given the high accuracy of both models (95.00% for ChatGPT and 93.33% for Gemini), they agree on the vast majority of problems, meaning the number of discordant pairs is expected to be very small. Because standard asymptotic chi-square approximations become mathematically inaccurate with small sample sizes, the exact binomial version of McNemar's test is required to calculate an accurate p-value.

(iii) Data Characteristics

For each problem, the paired results can be classified into one of four mutually exclusive cases, forming a 2×2 contingency table.

	Gemini Correct	**Gemini Incorrect**
ChatGPT Correct	53	4
ChatGPT Incorrect	3	0

Table 5: Contingency table for correctness outcomes.

The off-diagonal cells represent the discordant pairs: 4 problems were solved by ChatGPT but failed by Gemini, and 3 problems were solved by Gemini but failed by ChatGPT.

(iv) Test Computation

The exact McNemar's test evaluates whether the discordant pairs are equally distributed using a binomial distribution. Under H_0, we assume both models are equal, meaning the probability of either model winning a discordant pair is 50% ($p = 0.5$). This is analogous to flipping a fair coin to decide the winner of a disagreement.

The test statistic S represents the smaller number of discordant outcomes:

$$S = \min(\text{ChatGPT Only}, \text{Gemini Only}) = \min(4, 3) = 3$$

The exact two-sided p-value is calculated from the binomial distribution $\mathcal{B}(n = 4 + 3, p = 0.5)$. This calculates the exact mathematical probability of observing a 4-to-3 split (or a more extreme split) across the $n = 7$ total discordant cases, assuming the true probability is 0.5.

(v) Results

Test Statistic: $S = 3.0$
p-value (Exact): $p = 1.0000$
Significance Level (α): 0.05

(vi) Decision and Conclusion

- Since $p = 1.0000 > \alpha = 0.05$: **Fail to reject** H_0

Conclusion: There is no statistically significant difference in the correctness of SQL queries generated by ChatGPT Mini and Gemini Flash ($p = 1.0000$, exact McNemar's test). The observed difference (one additional accepted problem for ChatGPT) is negligible and easily attributable to random variation rather than a consistent performance advantage. Both models demonstrated highly comparable and reliable performance on this dataset.

3.4.2 Model's Execution Time Comparison

The third hypothesis concerns the execution time of accepted SQL queries and is formulated as follows:

(i) Hypothesis **Null Hypothesis (H_0):** *There is no statistically significant difference in the execution times of SQL queries generated by ChatGPT Mini and Gemini Flash.*

Alternative Hypothesis (H_1):
There is a statistically significant difference in the execution times of SQL queries generated by ChatGPT Mini and Gemini Flash.

The comparison is based exclusively on problems accepted by both models with available runtimes, resulting in a paired sample of $n = 53$ observations.

(ii) Statistical Test Selection

Since each problem was submitted to both models and the same evaluation platform was used, the observations are naturally paired. The choice of statistical test depends on the distributional properties of the paired differences $d_i = \text{ChatGPT}_i - \text{Gemini}_i$.

Data Assumption Checks:

- **Normality assessment**: The Shapiro-Wilk test was applied to the paired differences.
- **Visual inspection**: A Q-Q plot was generated to complement the formal normality test.

(iii) Data Characteristics

Normality Analysis:

The Shapiro-Wilk test yielded $W = 0.5037$ with $p < 0.0001$, providing strong evidence against the assumption of normality. The null hypothesis of the Shapiro-Wilk test (that the differences follow a normal distribution) is therefore rejected at $\alpha = 0.05$.

Test	**Statistic**	**p-value**
Shapiro-Wilk	$W = 0.5037$	$p < 0.0001$

Table 6: Shapiro-Wilk normality test on paired runtime differences.

The Q-Q plots in Figures 6 and 7 confirm this finding by visualizing the distribution of execution times for each model independently. The sample quantiles deviate substantially from the theoretical normal quantiles, particularly in the upper tails. This indicates heavy-tailed and heavily skewed non-normal behaviour caused by extreme outliers. Because the individual distributions (and their paired differences) are not normally distributed, parametric testing cannot be used.

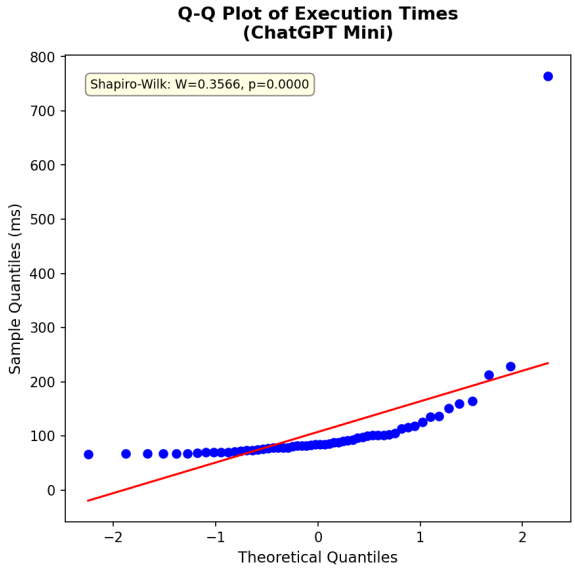

Figure 6: Q-Q plot of execution times for ChatGPT Mini.

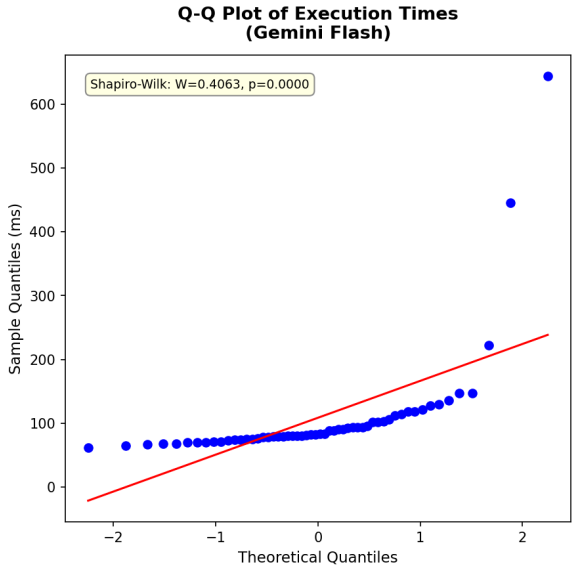

Figure 7: Q-Q plot of execution times for Gemini Flash.

Selected Test Evaluation: The **Wilcoxon signed-rank test** was chosen for this comparison. It is a non-parametric test for paired samples that does not require the differences to be normally distributed and is robust to skew and outliers. For each problem we compute the paired difference $d_i = \text{ChatGPT}_i - \text{Gemini}_i$, remove ties ($d_i = 0$), rank the absolute differences $|d_i|$ and assign the ranks the sign of the original differences. The test statistic W is the smaller of the sum of positive ranks and the sum of negative ranks; a small W indicates a systematic shift in one direction. When the number of non-zero pairs is small the exact distribution is used to obtain the p-value, otherwise a normal approximation is applied. The Wilcoxon signed-rank test therefore assesses whether the median paired difference differs from zero, i.e. whether one model is consistently faster than the other.

(iv) Test Computation

The Wilcoxon signed-rank test ranks the absolute values of the paired differences and computes the test statistic W as the smaller of the sum of positive ranks and the sum of negative ranks:

$$W = \min(W^+, W^-)$$

where W^+ is the sum of ranks associated with positive differences and W^- is the sum of ranks associated with negative differences.

For the observed data:

- Number of paired observations: $n = 53$
- Mean difference: $\bar{d} = 4.96$ ms

- Median difference: $\tilde{d} = 1.00$ ms

(v) Results

Test Statistic: $W = 700.5$
p-value: $p = 0.8943$
Significance Level (α): 0.05
Effect Size: Cohen's $d = 0.0415$ (negligible)

(vi) Decision and Conclusion

- Since $p = 0.8943 > \alpha = 0.05$: **Fail to reject** H_0

Conclusion: There is no statistically significant difference in the execution times of SQL queries generated by ChatGPT Mini and Gemini Flash ($W = 700.5$, $p = 0.8943$). The effect size (Cohen's $d = 0.0415$) is negligible, confirming that the two models produce queries of comparable runtime efficiency. This result is consistent with the descriptive analysis presented in the previous section, where both models exhibited very similar median runtimes and largely overlapping distributions.

3.4.3 Impact of Problem Difficulty on Correctness

(i) Hypothesis

This hypothesis addresses whether the difficulty level of the problem significantly influences the ability of Large Language Models to generate correct SQL queries.

Null Hypothesis (H_0): *The proportion of correct SQL queries generated by the models is independent of the problem difficulty.*

Alternative Hypothesis (H_1):
The proportion of correct SQL queries generated by the models depends significantly on the problem difficulty.

(ii) Statistical Test Selection

To answer this research question, the results from both models were combined to test the general effect of difficulty on LLM performance. The data forms a categorical contingency table relating difficulty (Easy, Medium, Hard) to correctness (Accepted, Not Accepted).

Data Assumption Checks:

- **Categorical Association:** The standard statistical approach for comparing categorical variables (e.g., Difficulty vs. Correctness) is the Chi-Square Test of Independence (χ^2).
- **Expected Frequencies & Sparse Data:** The Chi-Square test relies on large sample approximations and becomes inaccurate if there are cells in the contingency table with very

small expected frequencies (typically less than 5). Because the models achieved near-perfect performance on Easy and Medium problems, the number of incorrect outcomes for these categories is extremely low (0 and 1, respectively).

- **Test Adjustment:** To address this sparse data issue, Fisher's Exact Test was selected. Fisher's Exact Test serves as a specialized, exact version of the Chi-Square test designed specifically to handle small cell counts without relying on approximations. Furthermore, to accommodate the standard 2×2 implementation of Fisher's Exact Test, the difficulty levels were collapsed into two categories: "Hard" and "Not Hard" (combining Easy and Medium).

(iii) Data Characteristics

Combining the 60 problems evaluated by each model yields $n = 120$ total observations. The collapsed contingency table is as follows:

	Correct	**Incorrect**	**Total**
Not Hard (Easy + Medium)	79	1	80
Hard	34	6	40
Total	113	7	120

Table 7: Collapsed contingency table for correctness by difficulty.

The table shows that only 1 out of 80 submissions failed for Not Hard problems, whereas 6 out of 40 submissions failed for Hard problems.

(iv) Test Computation

Fisher's Exact Test calculates the exact probability of observing the given contingency table, or any table more extreme, assuming that the null hypothesis (independence) is true.

The two-sided p-value is computed from the hypergeometric distribution of the cell counts.

(v) Results

p-value (Fisher's Exact Test): $p = 0.03173$
Significance Level (α): 0.05

(vi) Decision and Conclusion

- Since $p = 0.03173 \leq \alpha = 0.05$: **Reject** H_0

Conclusion: Problem difficulty has a statistically significant influence on the correctness of SQL queries generated by the models ($p = 0.03173$, Fisher's Exact Test). Specifically, the models are significantly more likely to produce incorrect solutions when faced with Hard problems compared

to Easy and Medium problems. This confirms that while LLMs are highly reliable for basic and intermediate SQL generation, their performance meaningfully degrades as problem complexity increases.